

Problema A

Você deve implementar o algoritmo de **ordenação por inserção** (insertion Sort) em C. Este algoritmo ordena um vetor percorrendo-o da esquerda para a direita e, a cada iteração, insere o elemento atual na posição correta dentro da parte já ordenada do vetor. A explicação do algoritmo está no final desta especificação.

Sua tarefa é implementar a seguinte função:

```
void insertionSort(int vetor[], int n);
```

que integra o seguinte código:

```
#include <stdio.h>

void insertionSort(int vetor[], int n) {
    // Implemente a lógica da ordenação por inserção aqui
}

void imprimeVetor(int vetor[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d", vetor[i]);
        if (i < n - 1) {
            printf(" ");
        }
    }
    printf("\n");
}

int main() {
    int n;

    scanf("%d", &n);

    int vetor[n];
    for(int i = 0; i < n; i++){
        scanf("%d", &vetor[i]);
    }

    insertionSort(vetor, n);
    imprimeVetor(vetor, n);

    return 0;
}
```

Entrada

A primeira linha contém um número inteiro N ($1 \leq N \leq 1000$), representando a quantidade de números a serem ordenados. A segunda linha contém N números inteiros separados por espaço (podem ser positivos ou negativos).

Saída

Uma única linha contendo os N números ordenados de forma crescente, separados por um espaço. Deve haver uma quebra de linha ($\backslash n$) ao final da saída.

Exemplo de entrada	Exemplo de saída
5 4 2 8 1 3	1 2 3 4 8

Funcionamento do algoritmo de inserção inserindo 1 elemento

O algoritmo de ordenação por inserção funciona inserindo elemento a elemento em um vetor já ordenado. Por exemplo, considere o vetor abaixo:

10	15	18
----	----	----

Suponha que queremos inserir o valor 14 no vetor ordenado acima.

10	15	18	14
----	----	----	----

Comparamos o 14 elemento a elemento, da direita para a esquerda, até encontrar a sua posição correta. Começamos com o 18. Como 14 é menor que 18, então o 14 deve vir antes do 18 e trocamos o 18 com o 14.

10	15	14	18
----	----	----	----

Agora comparamos o 14 com o 15. Como 14 é menor que 15, então o 14 deve vir antes do 15 e trocamos o 15 com o 14.

10	14	15	18
----	----	----	----

Agora comparamos o 14 com o 10. Como o 14 é maior que 10, então ele está na sua posição correta, já que o vetor está ordenado. O resultado final então é o vetor ordenado:

10	14	15	18
----	----	----	----

Funcionamento do algoritmo de inserção ordenando um vetor

Para ordenar um vetor, o algoritmo de ordenação por inserção funciona dividindo a lista em duas partes, uma com elementos já ordenados e a outra com elementos ainda não ordenados. Assim, a cada iteração do algoritmo, o primeiro elemento da parte não ordenada é inserido em sua posição correta na parte ordenada. Ao final das iterações, a lista estará ordenada.

Considere o seguinte vetor:

80	20	30	50	10
----	----	----	----	----

Começamos com o vetor ordenado [80] e o vetor não ordenado [20, 30, 50, 10]. Vamos inserir o primeiro elemento do vetor não ordenado no vetor ordenado. Em outras palavras, vamos inserir o elemento 20 no vetor ordenado [80]. O método é exatamente o que usamos na seção anterior. Após a inserção do elemento 20, teremos o vetor ordenado [20, 80] e o vetor não ordenado [30, 50, 10]. De modo visual, temos:

80	20	30	50	10
----	----	----	----	----

20	80	30	50	10
----	----	----	----	----

20	30	80	50	10
----	----	----	----	----

20	30	50	80	10
----	----	----	----	----

10	20	30	50	80
----	----	----	----	----

Onde a parte verde é o vetor ordenado e a parte vermelha é a parte não ordenada.